Course Code	Course Title	Sem	Category	Periods/Week				C
				L	T	P	S	
CSB1321	**WEB TECHNOLOGY**	**VI**	**PC**	**3**	**0**	**0**	**3**	**3**
Pre-requisite	Web Application Development, Java Programming	**Course Type**	Theory	**QP TYPE**				QP2

Course Objectives:

The course aims to equip learners with the skills to design and develop dynamic web applications while exploring client-server architecture and communication protocols. Additionally, it covers web development using frameworks to enhance efficiency and scalability.

Course Outcomes:

Upon completion of the course, the students will be able to:

CO	Course Outcomes
CO1	Understand the fundamentals of web technologies and develop proficiency in designing and building static websites using HTML and CSS.
CO2	Gain practical experience in creating dynamic web pages with JavaScript, including form validation and event handling.
CO3	Explore real-world applications of server-side programming through the development of dynamic web applications using Servlets and JSP.
CO4	Develop dynamic Single Page Applications using AngularJS and its components.
CO5	Deploy dynamic web applications by integrating web application frameworks, NodeJS backend, ReactJS frontend, and Docker containerization.

SYLLABUS

Module I	**WEBSITE BASICS, HTML 5, CSS 3, WEB 2.0**	**9**

Web Essentials - HTTP Request Message - HTTP Response Message - Web Clients – Web Servers - HTML5 - Input Types – Page Structure Elements.

CSS3 - Selectors - Layouts - Inheritance - Transitions and Transformations - Animations.

Guided Self-Study Topics

HTML5 Advanced Features: Canvas API, Web Storage, and Geolocation.

Module II	**CLIENT-SIDE PROGRAMMING**	**9**

Java Script: An introduction to JavaScript - JavaScript DOM Model - Exception Handling – Validation.

Built-in Objects-Event Handling- DHTML with JavaScript- JSON introduction – Syntax – Function Files.

Guided Self-Study Topics		
JavaScript Advanced Concepts: Closures, Promises, and Async/Await.		
Module III	**SERVER-SIDE PROGRAMMING**	**9**
Servlets: Java Servlet Architecture- Servlet Life Cycle- Form GET and POST actions. Session Handling- Understanding Cookies- Database Connectivity: JDBC.		
Guided Self-Study Topics		
Session Management Techniques: Token-based authentication and secure session handling.		
Module IV	**FRONT-END FRAMEWORK DEVELOPMENT**	**12**
Introduction to Single Page Applications (SPAs), AngularJS, Directives in AngularJS, Controllers and Dependency Injection. Filters and Services, Angular Forms, outing and Navigation in AngularJS.		
Guided Self-Study Topics		
Web Security with PHP: Protecting against SQL injection and XSS.		
Module V	**FULL-STACK DEVELOPMENT TOOLS AND UI/UX**	**12**
Web Application Frameworks, Virtual Machines vs Containers, NodeJS: Environment Setup, Server Creation, Handling Requests, UI/UX Basics: Principles, Responsive Design, Wireframes. ReactJS: Components, Props, State, Docker: Containers, Dockerfile, Networking, Docker Compose.		
Guided Self-Study Topics		
Node Package Manager (NPM).		
		Total Hours: 45
Learning Resources:		
Text Books:		
1	Jeffrey C and Jackson, “Web Technologies A Computer Science Perspective”, Pearson Education, 2011.	
2	Paul Deitel, Harvey Deitel and Abbey Deitel, “Internet and World Wide Web”, Pearson Education, Fifth Edition, 2018.	
Reference Book7s:		
1	Godbole & Khate, “Web Technologies”, McGraw Hill Education, Third edition, 2017.	
2	UttamK.Roy, “Web Technologies”, Oxford University Press, 2011.	
3	Frank Zammetti, “Modern Full-Stack Development: Using TypeScript, React, Node.js, Webpack, and Docker”, Apress Publisher, 2020.	

Course Code	Course Title	Sem	Category	Periods/Week				C
				L	T	P	S	
CSB1322	**Compiler Design**	**VI**	**PC**	**2**	**1**	**0**	**3**	**3**
Pre-requisite	Theory of Computation, Computer Architecture and Organization	**Course Type**	Theory (PMBL)	**QP TYPE**				QP2

Course Objectives:

To explore the principles, algorithms, and data structures involved in the design and construction of compilers

Course Outcomes:

Upon completion of the course, the students will be able to:

CO	Course Outcomes
CO1	Explain the phases in compilation process and model a lexical analyzer
CO2	Construct different types of parsers for a given grammar and automation using YACC tool.
CO3	Design and implement intermediate code for efficient execution and optimization in compilers
CO4	Understand code optimization techniques and run-time environment.
CO5	Learn how to apply code optimization techniques and implement a simple code generator.

SYLLABUS

Module I	**INTRODUCTION TO COMPILERS AND LEXICAL ANALYSIS**	**9**

Compilers – Analysis of the source program – Phases of a compiler – Cousins of the Compiler – Grouping of Phases – Compiler construction tools – Lexical Analysis – Role of Lexical Analyzer – LEX – Input Buffering – Specification of Tokens – Recognition of Tokens.

Guided Self-Study Topics

Types of Compiler, Bootstrapping

Case Study Problems

Analyzing C source code to identify syntax and semantic errors

Mapping each phase of the compiler using a sample arithmetic expression.

Comparison of compiler with interpreter and assembler in a real-time application.

Implementing front-end vs back-end compiler stages for a mini programming language.

Using LEX to build a simple expression evaluator.

Designing a lexical analyzer for a subset of C keywords.

Writing a LEX program to identify integers, floating point numbers, and identifiers. Implementing two-buffer scheme for scanning large input files efficiently. Implementing Input Buffering with Lookahead Mechanism in Lexical Analysis Writing Regular Expressions to Specify Tokens for Keywords, Identifiers, and Operators		
Module II	**SYNTAX ANALYSIS**	**9**
Role of the parser – Writing Grammars – Context-Free Grammars – Top Down parsing – Recursive Descent Parsing – Predictive Parsing – Bottom-up parsing – Shift Reduce Parsing – Operator Precedence Parsing – LR Parsers – SLR Parser – Canonical LR Parser – LALR Parser – YACC		
Guided Self-Study Topics		
Error Recovery in LR Parser, Comparison of LR Parsers.		
Case Study Problems		
Create a grammar to represent arithmetic expressions. Develop a recursive descent parser for arithmetic expressions. Find the FIRST and FOLLOW sets for a context-free grammar. Construct a predictive parsing table for a given grammar. Simulate shift-reduce parsing for arithmetic expressions. Parse expressions using operator precedence parsing technique. Identify shift-reduce and reduce-reduce conflicts in SLR, CLR, and LALR parsers for ambiguous grammars. Construct an SLR parsing table for a given grammar and parse a sample input. Demonstrate LALR parsing by generating the table and parsing a simple expression. Compare output of SLR, LR, and LALR parser for the same input string.		
Module III	**INTERMEDIATE CODE GENERATION**	**9**
Intermediate languages – Declarations – Assignment Statements – Boolean Expressions – Case Statements – Procedure calls – Back patching		
Guided Self-Study Topics		
Graphical representations of Intermediate languages.		
Case Study Problems		
Convert an expression like a = b + c * d into three-address code. Write intermediate code for an if condition with one statement. Change variable declarations into intermediate form. Write three-address code for a while loop. Generate code for Boolean expressions using short-circuit evaluation. Use backpatching to implement code for an if-else condition.		

Create TAC for a switch-case block with break statements. Generate intermediate code for a simple function call with one parameter. Translate a nested if statement into intermediate code. Compare postfix and three-address code representations for a small expression.		
Module IV	**CODE OPTIMIZATION AND RUN TIME ENVIRONMENTS**	**12**
Introduction– Principal Sources of Optimization – Optimization of basic Blocks – DAG representation of Basic Blocks - Introduction to Global Data Flow Analysis – Runtime Environments – Storage Organization – Storage Allocation strategies – Access to non-local names – Parameter Passing, Error detection and recovery.		
Guided Self-Study Topics		
Machine dependent and machine independent optimizations, Local and global optimizations.		
Case Study Problems		
Remove dead code from a block of TAC. Apply constant folding to simplify constant expressions at compile time. Use DAG (Directed Acyclic Graph) to eliminate common subexpressions in a basic block. Identify live variables in a block and mark unnecessary statements. Demonstrate common subexpression elimination with example. Compare static and dynamic memory allocation with examples. Identify and explain the memory layout of a sample C program. Show parameter passing using call-by-value and call-by-reference. Simulate run-time stack creation for a recursive function call. Apply basic block optimization techniques on a given code block.		
Module V	**CODE GENERATION**	**12**
Issues in the design of code generator – The target machine – Basic Blocks and Flow Graphs – Next-use Information – A simple Code generator – Peephole Optimization.		
Guided Self-Study Topics		
Register Allocation and Assignment		
Case Study Problems		
Generate target code for an arithmetic expression using registers. Construct flow graph and identify basic blocks for a small code. Use next-use information to decide which variable to store in registers. Generate assembly code for a given three-address instruction. Simulate simple code generation for arithmetic expressions. Apply peephole optimization to remove redundant instructions.		

Apply strength reduction to optimize multiplication or division operations. Detect and eliminate unreachable code during code generation. Generate target code for relational expressions used in conditional statements. Compare code generation for stack-based vs register-based machines.	
Total Hours: 45	
Learning Resources:	
Text Books:	
1	Alfred V. Aho, Jeffrey D Ullman, “Compilers: Principles, Techniques and Tools”, Pearson Education Asia, 2008.
Reference Books:	
1	Allen I. Holub, “Compiler Design in C”, Prentice Hall of India, 2003
2	C. N. Fischer and R. J. LeBlanc, “Crafting a compiler with C”, Benjamin Cummings, 2003
3	Henk Alblas and Albert Nymeyer, “Practice and Principles of Compiler Building with C”, PHI, 2001.
4	Kenneth C. Louden, Compiler Construction – Principles and Practice, Cengage Learning Indian Edition, 2006.

Course Code	Course Title	Sem	Category	Periods/Week				C
				L	T	P	S	
CSB1323	**CRYPTOGRAPHY AND NETWORK SECURITY**	**VII**	**PC**	**3**	**0**	**2**	**3**	**4**
Pre-requisite	Computer Networks	**Course Type**	Theory cum Practical	**QP TYPE**				QP1

Course Objectives:

Understanding cryptographic theories, algorithms, and essential approaches is fundamental to building robust security systems. Various encryption and decryption techniques help safeguard sensitive information from potential attacks.

Course Outcomes:

Upon completion of the course, the students will be able to:

CO	Course Outcomes
CO1	Understand basic cryptographic algorithms, message and web authentication and security issues.
CO2	Demonstrate the ability to apply various cryptographic operations of symmetric cryptographic algorithms to ensure data security, confidentiality, and integrity
CO3	Apply the ability to utilize various cryptographic operations of public key cryptography to ensure secure communication, authentication, and data integrity
CO4	Determine the implementation of various authentication schemes to simulate and secure different applications.
CO5	Understand the knowledge of diverse security practices and system security standards to ensure robust protection of information systems.

SYLLABUS

Module I	**Introduction**	**6**

Security trends – Legal, Ethical and Professional Aspects of Security, Need for Security at Multiple levels, Security Policies – Model of network security – Security attacks, services and mechanisms – OSI security architecture – Classical encryption techniques - Foundations of modern cryptography-Social engineering attacks

Guided Self-Study Topics

Cryptanalysis , product cryptosystem

Case Study Problems		
Brute force attack, steganography		
Module II	**Symmetric Key Cryptography**	**6**
Introduction to Number theory - Finite fields-SDES – Block cipher Principles of DES – Block cipher design principles – Block cipher mode of operation – Evaluation criteria for AES – Advanced Encryption Standard-Quantum safe cryptography.		
Guided Self-Study Topics		
RC4,Key Distribution		
Case Study Problems		
Blowfish		
Module III	**Asymmetric Key cryptography**	**6**
Primes – Primality Testing –Factorization – Euler‘s totient function, Fermat‘s and Euler‘s Theorem – Chinese Remainder Theorem —RSA cryptosystem – Key distribution – Key management – Diffie Hellman key exchange -ElGamal cryptosystem –-Elliptic curve cryptography		
Guided Self-Study Topics		
Exponentiation and logarithm ,Elliptic curve arithmetic		
Case Study Problems		
ECC in Modern Payment System		
Module IV	**Data Integrity and User Authentication**	**6**
Authentication requirement – Authentication function – MAC – Hash function – Security of hash function and MAC – SHA –Digital signature and authentication protocols – DSS-E-Signatures in Legal Frameworks- Entity Authentication: Challenge Response protocols-Authentication applications – Kerberos, X.509-Passwordless Authentication		
Guided Self-Study Topics		
Biometrics, Passwords		
Case Study Problems		
Comparative Study of CMAC and HMAC in Mobile Banking Applications		
Module V	**Network and Internet Security**	**6**
Electronic Mail security – PGP, S/MIME – IP security – IPSec in IoT security-Web Security –Intruders – Malicious software – viruses – Firewalls-Cloud Firewalls-Firewall as a Service		

Guided Self-Study Topics
Mitigating Cross-Site Scripting (XSS)
Case Study Problems
Zero-Day Malware Threats.
Total Hours: 60
Practical Experiments:
1. Perform encryption, decryption using the following substitution techniques a. Ceaser cipher b. Playfair cipher c. Hill Cipher d. Vigenere cipher 2. Perform encryption and decryption using following transposition techniques a. Rail fence b. Row & Column Transformation 3. Apply DES algorithm for practical applications. 4. Apply AES algorithm for practical applications. 5. Implement RSA Algorithm using HTML and JavaScript. 6. Implement the SIGNATURE SCHEME – Digital Signature Standard.
Additional Experiments / Exercises:
1. Implement the Diffie-Hellman Key Exchange algorithm for a given problem. 2. Calculate the message digest of a text using the SHA-1 algorithm.
Mini Project (Sample)
1. Demonstrate intrusion detection system (ids) using any tool eg. Snort or any other s/w. 2. Automated Attack and Penetration Tools Exploring N-Stalker, a Vulnerability Assessment Tool 3. Defeating Malware i) Building Trojans ii) Rootkit Hunter 4. Efficient Network Intrusions Attacks Evidence Logging and Management
Learning Resources:
Text Books:

1	William Stallings, Cryptography and Network Security: Principles and Practice, PHI 8th Edition, 2006.

2	Behrouz A. Foruzan, Cryptography and Network Security, Tata McGraw Hill 2007.
3	C K Shyamala, N Harini and Dr. T R Padmanabhan: Cryptography and Network Security, Wiley India Pvt.Ltd
4	Wade Trappe, Lawrence C. Washington: Introduction to Cryptography with Coding Theory,3rd Edition, 2020.
Reference Books:	
1	Cryptography And Network Security Dr S.Bose PEARSON , First Edition Edition - 1 March 2016
2	Cryptography and Network Security by Atul Kahate, McGraw Hill Education, Edition. Third; July 2017
3	Network Security and Cryptography by Bernard Menezes, Cengage. Learning, 2010 edition

Course Code	Course Title	Sem	Category	Periods/Week				C
				L	T	P	S	
CSB1331	**WEB TECHNOLOGY LABORATORY**	**VI**	**PC**	**0**	**0**	**2**	**0**	**1**
Pre-requisite	Web Application Development	**Course Type**	Practical	**QP TYPE**				ASPR

Course Objectives:

The course aims to teach web technologies like HTML, CSS, PHP, and JDBC, along with Java Servlets and database integration. It also covers the development of dynamic web applications using modern frameworks such as AngularJS, Node.js, and React.

Course Outcomes:

Upon completion of the course, the students will be able to:

CO	Course Outcomes
CO1	Develop static and dynamic web pages using HTML5 and CSS3, applying styling, animations, and validations.
CO2	Implement interactive client-side features with JavaScript, utilizing DOM manipulation, event handling, and form validation.
CO3	Implement Java Servlets for form handling, session management, and database connectivity using JDBC to interact with databases.
CO4	Create web applications using PHP for form validation, program control, and XML data handling.
CO5	Build dynamic web applications with AngularJS, Node.js, and React, incorporating UI/UX principles for interactivity and performance.

SYLLABUS

LIST OF EXPERIMENTS / EXERCISES

1 **Creating a Basic Webpage Using HTML and CSS:**

a. Create a webpage with HTML describing your department. Use paragraph and list tags.
b. Apply CSS styles to differentiate keywords using colors. Also, apply font styling like italics, underline, and two different fonts for suitable words. Use header tags for headings
c. Create hyperlinks on the words “Wi-Fi” and “LAN” to link them to their Wikipedia pages.
d. Insert an image and create a hyperlink such that clicking on the image redirects the user to another page.

	e. Use CSS to change the background color of the webpage. At the bottom, create a link to navigate to the top of the page using an anchor link.
2	**Understanding HTTP Communication:** a. Use browser developer tools to inspect an HTTP Request Message and HTTP Response Message. b. Identify request methods (GET, POST, PUT, DELETE) and analyze response headers and status codes. **c.** Create a simple HTML form that sends a request to a server and observe the request in the browser's network tab.
3	**Interactive Form Handling Using JavaScript:** a. Create a registration form with fields (Name, Email, Phone Number, Password, Confirm Password). b. Use JavaScript DOM manipulation to dynamically update error messages and change the background color of fields on focus. c. Implement form validation to check required fields, email format, phone number length, and password strength. d. Use event handling to validate fields on input change and button click. e. Apply exception handling using try...catch to manage unexpected errors during form submission.
4	**Dynamic Content Loading and JSON Handling Using JavaScript** a. Use DHTML and JavaScript to dynamically add, remove, and modify content on a webpage. b. Create a JSON file containing details of students (Name, Age, Course) and load the data into an HTML table using JavaScript. c. Implement a search function to filter student records based on input. d. Use JavaScript functions to modify and update JSON data dynamically. e. Apply event handling to update the webpage when new data is added or removed.
5	Implementing a Java Servlet and Form Handling: a. Create a Java Servlet that processes a HTML form using GET and POST methods. b. Implement the Servlet Life Cycle methods (init(), service(), and destroy()). c. Use session handling to store and retrieve user details. d. Implement cookies to track user preferences and display them on the next visit.
6	**Database Connectivity Using JDBC in Servlets:** a. Establish a JDBC connection to a database using a Java Servlet. b. Create a user registration form that inserts data into a database using JDBC. c. Retrieve and display stored user details in a web page.

	d. Implement error handling for database connection failures and invalid queries
7	**PHP and Form Handling:** a. Write a PHP script that displays a greeting message on a webpage. b. Use variables in PHP to store and display user information such as name and age. c. Implement program control structures in PHP to validate the user's input. d. Use built-in functions in PHP to manipulate strings, numbers, and perform mathematical operations. e. Create a form with PHP validation for user inputs like email and password and display appropriate error messages for invalid inputs.
8	**XML and XSL:** a. Create an XML document containing structured data about books (Title, Author, ISBN, etc.). b. Define a Document Type Definition (DTD) to validate the structure of the XML document. c. Create an XML Schema to enforce rules and constraints on the XML data. d. Use an XML parser in PHP to read and display the contents of the XML document. e. Apply XSL to transform the XML data into a formatted HTML table.
9	**AngularJS and MVC Architecture:** a. Create an AngularJS application that implements MVC architecture. b. Use 'ng' attributes such as ng-app, ng-model, and ng-controller to bind data and structure the application. c. Implement expressions and data binding to display dynamic content in the view. d. Use conditional directives like ng-if and ng-show to control the visibility of elements. e. Apply style directives like ng-class and ng-style to dynamically change styles based on data.
10	**AngularJS Directives, Controllers, and Web Application Frameworks:** a. Create AngularJS controllers to manage the application's logic and interact with the view. b. Use filters to format and display data (e.g., number, date, lowercase, etc.). c. Implement forms using AngularJS, including validation and two-way data binding. d. Build a service in AngularJS to manage application data and interact with external resources. e. Set up a Node.js environment to run AngularJS applications and explore UI

	& UX principles in designing the application interface.
CONTENT BEYOND SYLLABUS	
1	**Design and develop a role-based web application using Firebase Authentication and Firebase Realtime Database/Firestore. Your application should support at least two types of users (e.g., Admin and Student). Implement the following features:** a. User registration and login using Firebase Authentication (Email/Password and Google Sign-In). b. Store and retrieve user-specific data in Firebase Realtime Database or Firestore. c. Implement role-based access where: Admins can view and manage all users/data. ii) Students can view only their own data and perform limited operations. d. Use JavaScript and Firebase SDK for frontend integration.
	Total Hours: 30

Course Code	Course Title	Semester	Category	Periods/Week				C
				L	T	P	S	
CSB1333	**COMPREHENSION**	**VI**	**PC**	**0**	**0**	**2**	**0**	**1**
Pre-requisite		**Course Type**	Practical	**Course Module**				DM

Course Objectives:

To enhance technical competence and problem-solving abilities in core computer science areas, thereby improving employability and readiness for industry and competitive exams.

Course Outcomes:

Upon completion of the course, the students will be able to:

Course Outcomes		MAX BTL
CO1	Apply programming concepts and data structures to solve computational problems effectively.	3
CO2	Design, analyze, and evaluate algorithms and formal language models for problem solving and theoretical computation.	4
CO3	Demonstrate an understanding of computer organization, operating systems, and their role in efficient hardware-software interaction.	4
CO4	Construct and query relational databases and apply principles of software engineering in system development and testing.	4
CO5	Analyze the components of compiler design and computer networks to optimize program execution and data communication.	5

Syllabus			MAX BTL
Module I	**Programming and Data Structures**	**6**	**3**

C programming basics - Control structures (loops, conditionals) - Functions, recursion, scope and storage classes - Arrays, strings, structures, unions - Pointers and dynamic memory allocation - Linked lists (singly, doubly, circular) - Stacks and queues (array and linked list implementations) - Trees (binary trees, BST, traversals, heaps) - Graphs (BFS, DFS, representation, spanning tree)

Module II	**Algorithms and Theory of Computation**	**6**	**4**

Time and space complexity, Big-O notation - Recurrence relations and solving techniques - Divide and conquer, greedy algorithms - Dynamic programming and backtracking - Graph algorithms (shortest path, MST, topological sort) - Complexity classes (P, NP, NP-complete, NP-hard) - Regular expressions and finite automata (DFA/NFA) - Context-free grammars and pushdown automata - Turing machines and undecidability

Module III	**Computer Organization and Operating Systems**	**6**	**4**

Logic gates, Boolean algebra, K-maps - Combinational circuits (adders, multiplexers, etc.) - Sequential circuits (flip-flops, registers, counters) - Machine instructions, addressing

modes - CPU organization, instruction pipelining - Memory hierarchy, cache mapping, virtual memory - I/O organization and DMA - Process, threads, CPU scheduling algorithms - Deadlocks, memory allocation, paging, file systems		
Module IV	**Databases and Software Engineering 6**	**4**
Entity-relationship model - Relational model and relational algebra - SQL queries, joins, views - Functional dependencies and normalization (1NF–BCNF) - Transactions, ACID properties - Concurrency control and recovery techniques - Indexing and file organization - Software development life cycle (SDLC) - Software design, testing, agile, project management		
Module V	**Compiler Design and Computer Networks**	**5**
Lexical analysis and tokenization - Syntax analysis and parsing techniques (LL, LR) - Syntax directed translation - Intermediate code generation - Code optimization and code generation - OSI and TCP/IP models - Network protocols: IP, TCP, UDP, DNS, HTTP - Routing algorithms (distance vector, link state) - Congestion control, flow control, switching techniques		
	Total Hours	**30**
Learning Resources:		
Reference Books		
1	Reema Thareja, *Data Structures Using C*, Second Edition, Oxford University Press, 2020.	
2	Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman**,** *Compilers: Principles, Techniques, and Tools*, Second Edition, Pearson Education, 2006.	
3	Abraham Silberschatz, Peter B. Galvin, Greg Gagne**,** *Operating System Concepts*, Tenth Edition, Wiley, 2018.	
4	Behrouz A. Forouzan, Data Communications and Networking, Fifth Edition, McGraw Hill, 2012.	
5	Korth, Silberchatz, Sudarshan, *Database System Concepts*, Sixth Edition, McGraw Hill, 2019.	
6	Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein**,** *Introduction to Algorithms*, Third Edition, MIT Press, 2009.	
7	Morris Mano, M.D. Ciletti**,** *Digital Design*, Fifth Edition, Pearson Education, 2013.	
8	Rajiv Chopra**,** *Software Engineering: Principles and Practice*, Second Edition, Narosa Publishing House, 2015.	
9	Y.N. Srikant, Priti Shankar**,** *The Compiler Design Handbook: Optimizations and Machine Code Generation*, Second Edition, CRC Press, 2007.	

Professional Elective II

List of Courses

Course Code	Course Title	Sem	Category	Periods/Week				C
				L	T	P	S	
CSC1371	**APPLICATION DEVELOPMENT**	**VI**	**PE**	**2**	**0**	**2**	**2**	**3**
Pre-requisite	Web Application Development, Java Programming	**Course Type**	**Theory (PRBL)**	**QP TYPE**				**QP1**

Course Objectives:

To Develop the skills to create native applications with basic GUI components for intuitive user interactions, build cross-platform applications with effective event-handling mechanisms, and integrate location services and data storage to enhance functionality. Additionally, learn to develop web applications with seamless database access for efficient data management.

Course Outcomes:

Upon completion of the course, the students will be able to:

CO	Course Outcomes
CO1	Gain proficiency in developing native applications with graphical user interface (GUI) components.
CO2	Learn to create hybrid applications incorporating basic event-handling mechanisms.
CO3	Implement cross-platform applications with integrated location services and data storage capabilities.
CO4	Analyze the design and development of cross-platform applications by examining GUI elements and event-handling functionality.
CO5	Understand and apply fundamental principles to enhance app performance, maintainability, UI/UX design, and basic security measures.

SYLLABUS

Module I	**BASICS OF MOBILE AND WEB APPLICATION DEVELOPMENT**	**6+6**

Introduction to Web and Mobile Application Development: Native, Hybrid, and Cross-Platform Apps.
Progressive Web Apps, Principles of Responsive Web Design.

Guided Self-Study Topics		
Application Programming Interfaces (APIs) in Mobile and Web Development, Mobile Backend as a Service (MBaaS) and Its Role in Modern App Development.		
Module II	**NATIVE APP DEVELOPMENT USING JAVA**	**6+6**
Native Web App, Scenarios to create Native App, Tools for creating Native App, Popular Native App Development Frameworks. Java & Kotlin for Android, Swift & Objective –C for iOS, Basics of React Native, Native Components, JSX, State, Props.		
Guided Self-Study Topics		
Flutter for Cross-Platform App Development-App Performance Optimization Techniques.		
Module III	**HYBRID APP DEVELOPMENT**	**6+6**
Hybrid Web Apps: Development Criteria, Tools, Challenges. Popular Frameworks like Ionic and Apache Cordova.		
Guided Self-Study Topics		
Comparison of Hybrid, Native, and Cross-Platform App Development- Progressive Web Apps (PWAs) and Their Role in Modern Web Development.		
Module IV	**CROSS-PLATFORM APPLICATION DEVELOPMENT USING REACT NATIVE**	**6+6**
Cross-Platform App Development, React Native Architecture, Layouts, Styling, and Navigation, Event Handling and Data Flow. Introduction to Flutter, Architecture, Widgets, Layouts and Styling, Event Handling and User Interaction.		
Guided Self-Study Topics		
State Management in Cross-Platform Applications.		
Module V	**ESSENTIAL APP PERFORMANCE AND MAINTENANCE**	**6+6**
App Performance, Debugging, Maintainability, UI/UX Essentials. App Security, Performance Optimization Techniques.		
Guided Self-Study Topics		
Security Considerations in Mobile and Web App Development.		
		Total Hours: 60 Hours
Learning Resources:		
Text Books:		
1	Head First Android Development, Dawn Griffiths, O'Reilly ,3rd edition, 2021.	

2	Apache Cordova in Action" by Raymond K. Camden, Manning, 2019.
Reference Books:	
1	Android Programming for Beginners" by John Horton, Packt Publishing, 2nd Edition.
2	Native Mobile Development" by Shaun Lewis and Mike Dunn.
3	Building Cross-Platform Mobile and Web Apps for Engineers and Scientists: An Active Learning Approach" by Pawan Lingras, Matt Triff, Rucha Lingras.
4	Apache Cordova 4 Programming" by John M. Wargo, 2015.
5	React Native Cookbook" by Daniel Ward, Packt Publishing, 2nd Edition.

Course Code	Course Title	Sem	Category	Periods/Week				C
				L	T	P	S	
ITC1384	**SOCIAL NETWORK SECURITY**	**VI**	**OE**	**3**	**0**	**0**	**3**	**3**
Pre-requisite	Computer Networks	**Course Type**	Theory	**QP TYPE**				QP1

Course Objectives:

The students can able to develop semantic web related simple applications, knowledge about the Privacy and Security issues in Social Networking, data extraction and mining of social networks.

Course Outcomes:

Upon completion of the course, the students will be able to:

CO	Course Outcomes
CO1	Develop semantic web related simple applications
CO2	Address Privacy and Security issues in Social Networking
CO3	Explain the data extraction and mining of social networks
CO4	Explain the data extraction and mining of social networks
CO5	Describe the applications of social networks

SYLLABUS

Module I	**Fundamentals of Social Networking**	**9**

Introduction to Semantic Web, Limitations of current Web, Development of Semantic Web, Emergence of

The Social Web, Social Network analysis, Development of Social Network Analysis, Key concepts and measures in network analysis,

Guided Self-Study Topics

Historical overview of privacy and security, Major paradigms, for understanding privacy and security

Module II	**Security Issues in Social Networks**	**9**

The evolution of privacy and security concerns with networked technologies, Contextual influences on privacy attitudes and behaviors, Anonymity in a networked world.

Guided Self-Study Topics

Account hijacking, Malware and phishing attacks

Module III	**Extraction and Mining in Social Networking Data**	**9**

Extracting evolution of Web Community from a Series of Web Archive, detecting communities in social networks, Definition of community, evaluating communities, Methods

for community detection and mining, Applications of community mining algorithms, Tools for detecting communities social network infrastructures and communities,		
Guided Self-Study Topics		
Big data and Privacy		
Module IV	**Predicting Human Behavior and Privacy Issues**	**9**
Understanding and predicting human behavior for social communities, User data Management, Inference and Distribution, Enabling new human experiences, Reality mining, Context, Awareness, Privacy in online Social networks, Trust in online environment.		
Guided Self-Study Topics		
What is Neo 4j, Nodes, Relationships, Properties.		
Module V	**Access Control, Privacy and Identity Management**	**9**
Understand the access control requirements for Social Network, Enforcing Access Control Strategies, Authentication and Authorization, Roles-based Access Control, Host, storage and network access control options, Firewalls, Authentication, and Authorization in Social Network, Identity & Access Management, Single Sign-on, Identity Federation, Identity providers and service consumers.		
Guided Self-Study Topics		
The role of Identity provisioning.		
	Total Hours	**45**
Learning Resources:		
Text Books:		
1	Peter Mika,“ Social Networks and the Semantic Web, First Edition, Springer 2007.	
2	Borko Furht,“Hand book of Social Network Technologies and Application, First Edition, Springer,2010.	
3	Learning Neo4j3.x “Second Edition By Jérôme Baton, Rik Van Bruggen, Packt publishing	
4	David Easley, Jon Kleinberg, “Networks, Crowds, and Markets: Reasoning about a Highly Connected Worldǁ, First Edition, Cambridge University Press, 2010.	
Reference Books:		
1	Easley D.K leinberg J.,“Networks, Crowds, and Markets–Reasoning about a Highly Connected World”, Cambridge University Press,2010.	
2	Jackson, Matthew O.,“Social and Economic Networks”, Princeton University Press,2008.	
3	Guandong Xu, Yanchun Zhang and LinLi,“Web Mining and Social Networking–Techniques And applications”, First Edition, Springer, 2011.	